

RISC-V v11.0.0 测试结果

HighTec RISC-V Development Platform 11.0.0 | Release Notes 功能验证

被测工具链: C:\HighTec\toolchains\riscv\v11.0.0 (clang 11.0.0 based on LLVM 21.1.8, built 2026-09-02)
测试主机: Windows x86-64 | 测试日期: 2026-09-04 | 许可证: RLM (C:\HighTec\licenses, riscv-llvm 节点锁定)

本文档按照 *releasenotes.pdf* (Release 11.0.0) 列出的新增特性, 逐项验证本机可测试的内容, 测试代码与命令详见 `RISCV_v11.0.0_TestItems_Guide.txt`。

[CCRM-115] Upgrade to LLVM 21.1.8

验证 clang / lld / llvm 工具版本。三个工具均报告基于 LLVM 21.1.8, 与 release notes 一致。

```
clang.exe --version -> HighTec clang version 11.0.0 (...) based on LLVM 21.1.8 built on 2026-09-02
ld.lld.exe --version -> HighTec LLD version 11.0.0 (...) based on LLVM 21.1.8 built on 2026-09-02
llvm-nm.exe --version -> ... based on LLVM 21.1.8 built on 2026-09-02
```

测试结果: **通过**。

[CCRM-123] Upgrade to Reprise Licensing Manager (RLM) 18

验证内嵌 RLM 客户端版本与许可证检出机制。通过设置 `RLM_DIAGNOSTICS` 触发 RLM 诊断文件, 直接读取到内嵌 RLM 版本号。

```
RLM Diagnostics at 09/04/2026 14:11
RLM version: 18.0BL1
RLM platform: x64_w4
OS version: 6.2
ISV name: hightec-rt
Product: riscv-llvm v2026.12 OK
```

正常编译 (`RLM_LICENSE=C:\HighTec\licenses`) 许可证检出成功, 退出码 0; 将 `RLM_LICENSE` 指向无效路径时报错 `Can't read license data (-102)`, 机制正常。

测试结果: **通过** (RLM 18.0BL1 确认)。

[CCRM-169] Upgrade to Picolibc C library 1.8.11

检查 C 库头文件版本宏, 与 release notes 一致。

```
riscv\include\picolibc.h:
#define __PICOLIBC_VERSION__ "1.8.11"
#define _NEWLIB_VERSION      "4.3.0"
```

测试结果: **通过**。

[CCRM-176] Emit functions and static allocated variables into unique sections by default

11.0.0 中 `-ffunction-sections` / `-fdata-sections` (以及 `-fpragma-function-sections` / `-fpragma-data-sections`) 默认使能, 函数与静态分配变量默认生成独立节。不使用任何 `-ffunction-sections/-fdata-sections` 选项直接编译, 检查目标文件节名:

```
// sections_default.c / rodata_test.c (默认编译)
llvm-objdump -h sections_default.o ->
    .text.func_a      .text.func_b      .text.helper      (函数)
    .data.global_init .data.static_init  (已初始化数据)
    .bss.global_zero  .bss.static_zero   (零初始化数据)
    .rodata.table      .rodata.msg       (常量数据)
```

通过 `clang -###` 确认驱动默认传递 `-ffunction-sections -fdata-sections -fpragma-function-sections -fpragma-data-sections`。

注意事项：默认独立节会改变输出段的组织方式，用户工程链接脚本需包含 `*(.text*)`、`*(.data*)`、`*(.bss*)` 等带通配符的匹配规则，否则内容可能被遗漏。对自定义段（`#pragma clang section`）导致的问题，可通过 `-fno-pragma-function-sections -fno-pragma-data-sections` 关闭，迁移方法参考 HighTec 移植指南（与 TriCore-llvm v11.0.0 一致）。

测试结果：**通过**。

[CCRM-176] Garbage collection of unreferenced sections by the linker is enabled by default

11.0.0 中链接器默认开启未引用节回收（经 `clang` 驱动默认传递 `--gc-sections`）。通过独立未使用的函数/变量验证：

```
// gc_default.c
int used_global = 42;    int unused_global = 99;    // unused_global 不被引用
int used_func(int x){ return x + used_global; }
int unused_func(int x){ return x * unused_global; } // unused_func 不被引用
void _start(void){ (void)used_func(1); for(;;); }
```

经 `clang` 驱动默认链接后，用 `llvm-nm` 检查最终 ELF：

```
clang ... -nostdlib -T gc_default.ld gc_default.o -o gc_default_clang.elf
llvm-nm gc_default_clang.elf -> 仅 _start / used_func / used_global
                                unused_func / unused_global 被回收
(-### 确认驱动默认传 --gc-sections)
```

对照：直接调用 `ld.lld` 且不带 `--gc-sections` 时未引用符号保留（符合 `ld` 原生语义）；显式 `--gc-sections` 时回收，与 `clang` 驱动默认行为一致。

测试结果：**通过**。

[CCRM-110] Add option -Wsection-feature to warn when a function or global variable is affected by more than one section defining feature

当同一个函数/变量被赋予多个段特性（`__attribute__((section))`）与 `#pragma clang section`）时，开启该选项后编译器给出警告。

```
// section_feature.c (按 userguide 示例)
#pragma clang section bss=".test1"
__attribute__((section(".test11"))) int test1;    // 警告
#pragma clang section bss=""
#pragma clang section text=".test2"
__attribute__((section(".test22"))) void test2(void){} // 警告
#pragma clang section text=""
#pragma clang section bss=".test3"
```

```
#pragma clang section bss=".test33" int test3;      // 警告（pragma 覆盖）
#pragma clang section bss=""
```

```
clang -Wsection-feature -c section_feature.c -> 4 条警告，例如：
warning: .test11 uses more than one section feature [-Wsection-feature]
warning: '#pragma clang section' overrides previous active section
clang -Wno-section-feature ... -> 无警告（可抑制）
clang（默认，不带选项） -> 无警告（选项默认关闭）
```

实测附加行为：由于 `-ffunction-sections/-fdata-sections` 默认开启，单独的 `__attribute__((section))` 也会被判定为“超过一个 section 特性”而告警（即使加 `-fno-function-sections -fno-data-sections` 亦然），属本工具链实现行为，使用该选项时需注意。

测试结果：**通过**。

[CCRM-150] Add option `-fno-zero-initialized-in-bss-gcc` as a GCC compatible variant of `-fno-zero-initialized-in-bss`

默认情况下全局/静态变量初始化为 0 或不初始化都放入 `.bss`。GCC 中 `-fno-zero-initialized-in-bss` 用于区分显式 `=0` 与未初始化变量；为保证 GCC 兼容性，HighTec 新增 `-fno-zero-initialized-in-bss-gcc` 变体（仅把显式 `=0` 的变量移出 `.bss`，隐式未初始化变量仍留在 `.bss`）。

```
// bss_test.c
int implicit_zero;      // 隐式零初始化
int explicit_zero = 0;  // 显式零初始化
int nonzero = 7;        // 非零对照
```

```
llvm-nm -S obj\bss_*.o (B=\.bss 节, D=\.data 节)
变量      默认      -fno-zero-init-in-bss  -fno-zero-init-in-bss-gcc
implicit_zero  B          D          B
explicit_zero  B          D          D
nonzero        D          D          D
```

三组行为与 `userguide` 表格完全一致（`-gcc` 变体仅把显式 `=0` 移到 `.data`，匹配 GCC 行为）。

测试结果：**通过**。

[TCGEN-241] Eliminated dependency of the toolchain binaries to the MSVC 2015 redistributable on Windows

用 `llvm-objdump -p` 检查全部工具链可执行文件/DLL 的导入依赖，确认不含 MSVC 2015 再发行运行库。

```
clang.exe / ld.lld.exe / llvm-nm.exe / llvm-objcopy.exe / clangd.exe / libclang.dll
导入 DLL 均为 Windows 系统 DLL：
  KERNEL32, ADVAPI32, ntdll, WS2_32, USER32, OLEAUT32,
  bcrypt, WINHTTP, CRYPT32, IPHLPAPI, VERSION
未发现 msvcrt140.dll / vcruntime140.dll / msvcr*.dll
```

测试结果：**通过**。

[附加] 端到端冒烟测试（整体 toolchain 可用性）

使用 `picolibc` 完整编译+链接一个可执行 ELF，验证 `clang / lld / crt0 / C 库` 全链路可用。

```
// main.c（使用 printf/memcpy 等 picolibc 库函数）+ syscalls.c（裸机 read/write/lseek/close stub）
```

```
clang -target riscv32-unknown-elf -march=rv32imafdc -mabi=ilp32 \
-T ...\\riscv\\lib\\rv32imafdc\\ilp32\\except\\picolibc.ld \
-L ...\\except crt0.o main.c syscalls.c -lc -lm -o main.elf

llvm-readelf -h main.elf -> ELF32 / Machine: RISC-V / Entry: 0x80000000
llvm-size main.elf -> text 34752 data 96 bss 4624
llvm-nm main.elf -> 含 _start / main / printf (来自 picolibc) / add
```

测试结果：**通过**。

[附加] 累积特性抽测 (1.1.0 ~ 1.3.0)

特性	命令	结果
目标 CPU ifx-rc1-base-h-s (TCRV-181)	-mcpu=ifx-rc1-base-h-s	编译通过
目标 CPU ifx-rc1-perf-h-rt (TCRV-202)	-mcpu=ifx-rc1-perf-h-rt	编译通过
rv32e 扩展 (TCRV-195)	-march=rv32e -mabi=ilp32e	编译通过 (带 -Wnon-hightec 提示, rv32e 不在 HighTec 支持 multilib 列表, 属正常)
rv64 -mcmmodel=large (TCRV-179)	-march=rv64imafd -mabi=lp64d -mcmmodel=large	编译通过
multilib 增删 (TCRV-221)	检查 riscv\\lib 目录	与 1.3.0 清单一致

测试总结

- 11.0.0 新增特性中, 本机可测试项 (LLVM 21.1.8、RLM 18、Picolibc 1.8.11、默认独立节、默认 GC、-Wsection-feature、-fno-zero-initialized-in-bss-gcc、去除 MSVC2015 依赖、端到端冒烟、累积特性抽测) **全部测试通过**。
- 默认开启 `-ffunction-sections/-fdata-sections/--gc-sections` 属行为变更, 会对用户工程有影响: 链接脚本需匹配 `*(.text*)` 等通配规则; 自定义段问题可用 `-fno-pragma-function-sections -fno-pragma-data-sections` 处理。
- 本机无法测试项:** CCRM-10 (ARM64 Linux 主机支持, 需对应平台实测)、CCRM-78 (SPDX 3.0.1 FOSS 报告, 未随安装包落盘, 需官方门户获取)、CCRM-85 / TCTRIC-1348 / TCTRIC-1349 (安装器改进, 需交互运行安装器验证)。